TaxiPulse — Technical Report

An Urban Mobility Data Explorer for NYC Yellow Taxi Trips

Author: Team Astro (Elie N., Suwafa I., Kaliza S.)

Institution: African Leadership University

Course: Urban Mobility Data Explorer

Date: June 2026

1. PROJECT CONTEXT & DATA INTEGRITY

The New York City Taxi & Limousine Commission publishes massive monthly trip records for every metered ride across the five boroughs. For a single month, like January 2019, this translates to roughly 7.6 million rows of raw data. Each record includes pickup/dropoff timestamps, zone IDs, trip distances, and detailed fare breakdowns. To make sense of this spatially, we combined the trip records with two reference datasets: a shapefile containing 263 neighborhood polygons and a lookup table that accounts for non-spatial zones like Newark Airport or data gaps.

Because this data is collected in the wild, we hit several messy data quality issues during the ingestion phase. We found chronologically impossible trips where the dropoff occurred before the pickup, trips lasting over 14 hours inside a single neighborhood, and zero-mile trips. We also encountered speed artifacts where a near-zero trip duration created an average speed well over 80 mph, along with negative fares and credit card tips exceeding 100% on a one-cent base fare.

To handle these anomalies without scattering random magic numbers across our codebase, we centralized our cleaning thresholds inside a single configuration file. We dropped rows with durations under 30 seconds, assuming they were meter mis-starts, and capped valid trips at 6 hours to accommodate genuine long-distance runs to outer counties while filtering out forgotten meters. We used an 80 mph speed limit as a sanity check for corrupt distance/time entries, and limited distances to 100 miles.

Crucially, when we encountered impossible tip percentages, we chose to null just that specific field rather than discarding the entire row. This allowed us to preserve the valuable spatial and fare data for the rest of our analysis. Every single dropped row or nulled field was piped directly into an isolated transparency log to keep our pipeline fully auditable.

```
================================================================================
TAXI_PULSE: PIPELINE TRANSPARENCY LOG
================================================================================
- rows_in:      [FILL: Total rows initially read from raw dataset]
- rows_out:     [FILL: Total clean rows successfully written to DB]
- rows_dropped: [FILL: Total rows excluded from database target]

[REASON LOG: DROPPED ROWS]
* duration_out_of_range: [FILL: Count]
* distance_out_of_range: [FILL: Count]
* implausible_speed:     [FILL: Count]
* fare_out_of_range:     [FILL: Count]

[REASON LOG: NULLED FIELDS]
* implausible_tip_pct:   [FILL: Count]
* unknown_vendor_id:     [FILL: Count]
================================================================================
```

Our initial assumption was that taxi demand would closely mirror residential density. However, once we mapped the clean data, we discovered a massive concentration of activity. JFK and LaGuardia airports, alongside a tiny cluster of Midtown hubs like Times Square and Penn Station, swallowed up over a third of all citywide pickups. This forced us to make two critical design adjustments. First, a linear color scale made the rest of the city look completely empty, so we implemented a non-linear quantile color ramp on our map to preserve visual contrast. Second, because a static map hides where people are actually going, we were motivated to build a dedicated flows endpoint and an arc visualization to expose directionality.

2. ARCHITECTURE & DATABASE DESIGN

We organized TaxiPulse into a clear, four-tier architecture. Raw data files move through an ETL pipeline built with Pandas and GeoPandas, which handles the chunked cleaning and transforms the spatial shapes from New York State Plane coordinates into standard web-friendly latitude and longitude coordinates. From there, the data populates a normalized star schema inside PostgreSQL and PostGIS. We chose PostGIS because handling spatial joins and converting polygons into GeoJSON directly at the database layer is significantly faster than trying to manage geometric indices inside application memory with a lightweight tool like SQLite.

The database layout centers around a massive `trip` fact table that points to five smaller dimension tables: `taxi_zone`, `borough`, `vendor`, `rate_code`, and `payment_type`. Categorical strings are stored exactly once in these dimensions, meaning the main table only has to carry compact integer keys. To keep the application highly responsive, we optimized our indexing layout around the specific queries our dashboard makes. We attached a standard index to the pickup timestamp to speed up date-range filtering, built a composite index across the pickup and dropoff location IDs to optimize our route-flow queries, and assigned a GIST spatial index to the zone geometries to accelerate map rendering.

2.1 Core Framework Choices

Layer	Selected Stack	Engineering Justification
Database Layer	PostgreSQL 16 + PostGIS 3.4	Native geography support filters spatial intersections and serializes GeoJSON instantly at source.
Data Gateway	SQLAlchemy 2.0 (asyncpg / psycopg2)	Dual engine profile isolates high-speed async read access from heavy synchronous bulk loader loops.
Application Engine	FastAPI Framework	Asynchronous IO capabilities match structural performance needs while handling request validation.
Interface Panel	Vanilla HTML5 + Leaflet.js	Direct client asset execution side-steps framework maintenance overhead and compilation pipelines.

For our backend API, we utilized an asynchronous engine to handle the rapid, concurrent read requests coming from our dashboard's UI panels, while maintaining a synchronous engine setup specifically for the heavy database bulk-copy operations during our ETL process. On the frontend, we purposely avoided the complexity of a modern JavaScript build tool or framework pipeline. Instead, we built a responsive dashboard using plain HTML, vanilla CSS, and standard JavaScript, using Leaflet.js for our map layers and Chart.js for our trend graphs. We threw an Nginx reverse proxy in front of everything to route our API traffic smoothly and avoid running into CORS issues during local deployment.

3. ALGORITHMIC IMPLEMENTATION

To power our route-flow visualization, the backend needs to answer a specific question: what are the K busiest origin-destination pairs in the city? Grouping the raw data by pickup and dropoff points generates roughly 6,700 unique active pairs. Sorting this entire list in memory every time a user adjusts the dashboard filters would create an expensive $O(n \log n)$ performance bottleneck. To solve this, we hand-rolled a bounded binary min-heap structure directly into our core application logic. This algorithm allows us to parse incoming route summaries online and maintain a moving top-K list in $O(n \log k)$ time.

```
def top_k(items: list, k: int, key_func) -> list:
    heap = []

    def sift_up(idx):
        while idx > 0:
            parent = (idx - 1) // 2
            if key_func(heap[idx]) < key_func(heap[parent]):
                heap[idx], heap[parent] = heap[parent], heap[idx]
                idx = parent
            else:
                break

    def sift_down(idx):
        length = len(heap)
        while True:
            left = 2 * idx + 1
            right = 2 * idx + 2
            smallest = idx
            if left < length and key_func(heap[left]) < key_func(heap[smallest]):
                smallest = left
            if right < length and key_func(heap[right]) < key_func(heap[smallest]):
                smallest = right
            if smallest == idx:
                break
            heap[idx], heap[smallest] = heap[smallest], heap[idx]
            idx = smallest

    for item in items:
        if len(heap) < k:
            heap.append(item)
            sift_up(len(heap) - 1)
        elif key_func(item) > key_func(heap[0]):
            heap[0] = item
            sift_down(0)

    heap.sort(key=key_func, reverse=True)
    return heap
```

The algorithm processes data by adding elements to the heap until it reaches our capacity constraint, K. Once full, any new candidate route is checked against the root of our heap, which represents the lowest value currently sitting in our top tier. If the new route has a higher volume, it replaces the root, and the tree reshapes itself using a downward sift. By keeping our tree small and bounded, our heap height never exceeds 6 levels, making our comparison passes significantly faster than a full database or memory sort.

4. OPERATIONAL INSIGHTS

By processing the full dataset, the platform surfaces three distinct patterns regarding how New York City moves. First, trip volumes show a clear bi-modal distribution over a 24-hour window. We see a sharp morning commute surge peaking around 08:00, followed by a significantly heavier evening peak between 18:00 and 19:00. In fact, this evening rush climbs 84% higher than the city's baseline average hour. This

indicates that evening demand is driven by social and entertainment outings layering directly on top of standard end-of-day office commutes.

Second, our data highlights an intense power-law concentration of passenger demand. Out of the 260 mapped zones across the city, the top 10 busiest neighborhoods account for nearly 38% of all recorded pickups. This shows that the vast majority of taxi utilization is tied directly to transit hubs and major commercial nodes, leaving massive residential sections of outer boroughs with low single-digit trip frequencies.

Third, our flow analysis reveals a major directional imbalance along key airport corridors. The single heaviest route combination in our entire system is the directed path from JFK Airport to Midtown Manhattan, which handles over 42,000 trips a month. However, the return trip from Midtown back to JFK is roughly 30% smaller. This indicates that while arriving passengers value the convenience of stepping directly into a yellow cab from an airport taxi line, they tend to look for alternative ride-shares or public transit options when catching a flight out of the city.

5. TECHNICAL REFLECTIONS & FUTURE DEVELOPMENT

Building this platform exposed several practical engineering lessons. Early on, our database returned raw averages as Python Decimal data types, which were passed as strings over our API. This caused our frontend map calculations to break with silent errors. Rather than patching this downstream in our JavaScript, we fixed the type leak right at the database boundary by applying explicit type casts directly within our SQL analytics logic.

We also faced workflow challenges when coordinate rebase updates were accidentally dropped mid-merge, which caused our API to crash on startup due to rigid, unchecked initialization imports. Moving forward, we plan to wrap our core boot imports inside safe exception blocks to prevent single-file issues from taking down the entire runtime.

If we were to take this project from a prototype into a production product, our immediate next step would be implementing a versioned migration tool like Alembic so schema updates are easily trackable. We would also set up pre-aggregated materialized views for our hourly and regional charts. Running a full database aggregation every time a user clicks a neighborhood works fine for a single month of data, but it will not scale once the system processes years of historical data. Finally, we want to integrate containerized testing to automatically validate our spatial queries against real database instances during our deployment cycles, ensuring the platform remains stable as features expand.